

08 The Application Layer

08.04 Browser

Brief Internet Browser History

- Web browsers are the most widely used software
- There are five major browsers used on desktop today: Chrome, Internet Explorer, Firefox, Safari and Opera.
- On mobile, the main browsers are Android Browser, iPhone, Opera Mini and Opera Mobile, UC Browser, the Nokia S40/S60 browsers and Chrome – all of which, except for the Opera browsers, are based on WebKit

Browser Timeline

- **1990** – The WorldWideWeb (not to be confused with the World Wide Web) was the first browser ever created by W3C Director Tim Berners-Lee, then renamed Nexus to differentiate from the actual World Wide Web. Unlike today, this was the only browser and the only way to access the web.
- **1992** – Lynx was a texted-based browser that couldn't display any graphic content.
- **1993** – Mosaic was the first browser to allow images embedded in text making it "the world's first most popular browser".
- **1994** – A noticeable improvement to Mosaic came Netscape Navigator.

Browser Timeline

- **1995** – Internet Explorer made its debut as Microsoft's first web browser.
- **1996** – Opera started as a research project in 1994 that finally went public two years later. This was also arguably the beginning of the browser wars, mainly between IE 3 and Navigator 3 as Internet Explorer inched ahead with new capabilities.
- **2003** – Apple's Safari browser was released specifically for Macintosh computers instead of Navigator.
- **2004** – Mozilla launched Firefox as Netscape Navigator faded out.

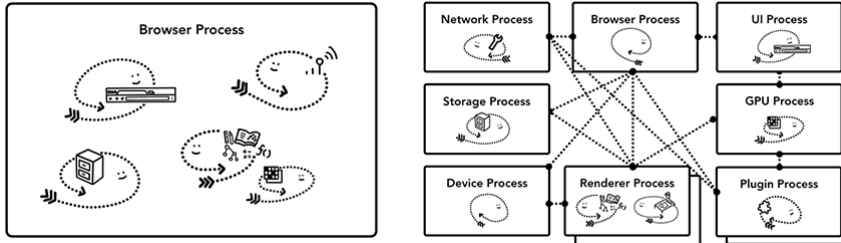
Browser Timeline

- **2007** – Mobile Safari was introduced as Apple's mobile web browser and continues to dominate the iOS market.
- **2008** – Google Chrome appeared to soon take over the browser market.
- **2011** – Opera Mini was released to focus on the fast-growing mobile browser market.
- **2015** – Microsoft Edge was born to combat Google.

The browser's main functionality

- The main function of a browser is to present the web resource you choose, by requesting it from the server and displaying it in the browser window.
- The location of the resource is specified by the user using a URI (Uniform Resource Identifier)

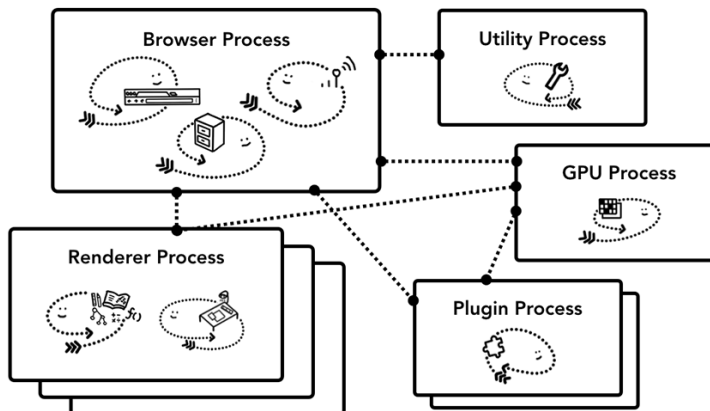
Browser Architecture



Different browser architectures in process/thread diagram

- The important thing to note here is that these different approaches are implementation details.
- There is no standard specification on how one might build a web browser.

Chrome's recent architecture



- At the top is the browser process coordinating with other processes that take care of different parts of the application.

Process and What it controls

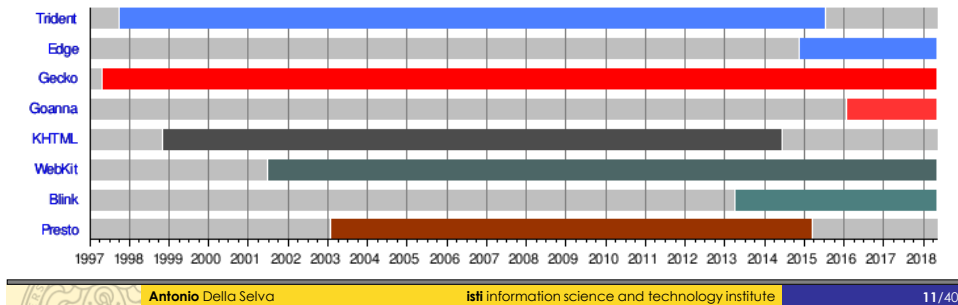
Browser	Controls "chrome" part of the application including address bar, bookmarks, back and forward buttons. Also handles the invisible, privileged parts of a web browser such as network requests and file access.
Renderer	Controls anything inside of the tab where a website is displayed.
Plugin	Controls any plugins used by the website, for example, flash.
GPU	Handles GPU tasks in isolation from other processes. It is separated into different process because GPUs handles requests from multiple apps and draw them in the same surface.

Layout / Rendering Engine

- It's able to render the content of given URL in browser screen and interprets the HTML, XML and CSS. It is single threaded.
- By default, It displays data according to your specified content type (MIME). For Example HTML, Images, XML, CSS, JSON, PDF etc.
- Key operation of Rendering engine is HTML Parser..

Notable browser engine

- Each browser use various engines like Chrome and Opera uses Blink, Firefox uses Gecko, IE Edge uses EdgeHTML, Internet Explorer uses Trident, Apple Safari uses WebKit.



What is the flow

1. Parsing HTML document by HTML Parser convert elements to Node and create Content Tree.
2. Parsing Styles code / document by CSS Parser and create Render Tree.
3. Render Tree goes through Layout Process. Element's Node get position coordinates.
4. Render Tree will be traversed and each node will be painted using the UI Back-end Layer.

Security and Sandboxing

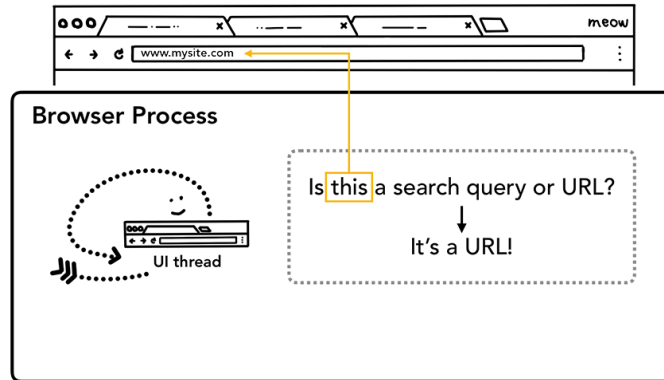
- Since operating systems provide a way to restrict processes' privileges, the browser can sandbox certain processes from certain features.
 - For example, the Chrome browser restricts arbitrary file access for processes that handle arbitrary user input like the renderer process.
- Because processes have their own private memory space, they often contain copies of common infrastructure. This means more memory usage as they can't be shared the way they would be if they were threads inside the same process.
- In order to save memory, Chrome puts a limit on how many processes it can spin up.

Saving more memory - Servicification in Chrome

- Chrome is undergoing architecture changes to run each part of the browser program as a service allowing to easily split into different processes or aggregate into one.
- General idea is that
 - when Chrome is running on powerful hardware, it may split each service into different processes giving more stability;
 - if it is on a resource-constraint device, Chrome consolidates services into one process saving memory footprint.

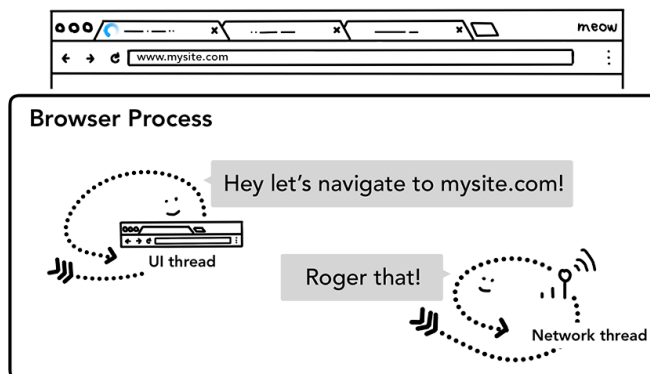
Navigation: handling input

When a user starts to type into the address bar, the first thing UI thread asks is "Is this a search query or URL?". In Chrome, the address bar is also a search input field, so the UI thread needs to parse and decide whether to send you to a search engine, or to the site you requested.



Navigation: start navigation

When a user hits enter, the UI thread initiates a network call to get site content. Loading spinner is displayed on the corner of a tab, and the network thread goes through appropriate protocols like DNS lookup and establishing TLS Connection for the request.



Navigation: Read Response

- Once the response body (payload) starts to come in, the network thread looks at the first few bytes of the stream if necessary. The response's Content-Type header should say what type of data it is, but since it may be missing or wrong, **MIME Type sniffing** is done here.
- This is a "tricky business" because we need to balance compatibility concerns with security issues.

Survey on detecting Content-Type

- HTML payload, no Content-Type header:
 - IE 7: Render as HTML
 - Firefox 2: Render as HTML
 - Safari 3: Render as HTML
 - Opera 9: Render as HTML
 - Chrome: Render as HTML
- HTML payload, Content-Type: "text/plain":
 - IE 7: Render as HTML
 - Firefox 2: Render as text
 - Safari 3: Render as text (Note: Safari will Render as HTML if the URL has an HTML extension)
 - Opera 9: Render as text
 - Chrome: Render as text

Survey on detecting Content-Type

- HTML payload, Content-Type: "application/octet-stream":
 - IE 7: Render as HTML
 - Firefox 2: Download as application/octet-stream
 - Safari 3: Render as HTML
 - Opera 9: Render as HTML
 - Chrome: Render Download as application/octet-stream
- GIF payload, Content-Type: "application/octet-stream":
 - IE 7: Render as GIF
 - Firefox 2: Download as application/octet-stream (Note: Firefox will Download as GIF if the URL has an GIF extension)
 - Safari 3: Download as Unknown (Note: Safari will Render as GIF if the // URL has an GIF extension)
 - Opera 9: Render as GIF
 - Chrome: Download as application/octet-stream

Navigation: Read Response

- This is also where the SafeBrowsing check happens. If the domain and the response data seems to match a known malicious site, then the network thread alerts to display a warning page.
- Additionally, **Cross Origin Read Blocking (CORB)** check happens in order to make sure sensitive cross-site data does not make it to the renderer process.

Navigation: Read Response

- Once all of the checks are done and Network thread is confident that browser should navigate to the requested site, the Network thread tells UI thread that the data is ready.
- UI thread then finds a renderer process to carry on rendering of the web page.
- Since the network request could take several hundred milliseconds to get a response back, an optimization to speed up this process is applied:
 - the UI thread tries to proactively find or start a renderer process in parallel to the network request.

Navigation: Commit Navigation

- Now that the data and the renderer process is ready, an IPC is sent from the browser process to the renderer process to commit the navigation.
- Once the browser process hears confirmation that the commit has happened in the renderer process, the navigation is complete and the document loading phase begins.
- At this point, address bar is updated and the security indicator and site settings UI reflects the site information of the new page.
- The session history for the tab will be updated so back/forward buttons will step through the site that was just navigated to.

Security Threats: Zero-day exploit

- A zero-day (or zero-hour) attack or threat is a computer threat that tries to exploit computer application vulnerabilities that are unknown to others, undisclosed to the software vendor, or for which no security fix is available.
- Zero-day exploits are used or shared by attackers before the software vendor knows about the vulnerability.
- When a vendor becomes aware of a security hole, there is a race to close it before attackers discover it or the vulnerability becomes public.
- A "zero day" attack occurs on or before the first or "zeroth" day of vendor awareness, meaning the vendor has not had any opportunity to disseminate a security fix to users of the software.

Security Threats: Clickjacking

- Is a malicious technique of tricking Web users into revealing confidential information or taking control of their computer while clicking on seemingly innocuous Web pages.
- A vulnerability across a variety of browsers and platforms, a clickjacking takes the form of embedded code or script that can execute without the user's knowledge, such as clicking on a button that appears to perform another function.
- Clickjacking, also known as UI redressing, is possible not because of a software bug, but because seemingly harmless features of Web pages can perform unexpected actions.

Security Threats: Cross Site Scripting

- XSS is a common attack in which an attacker injects a malicious piece of code into an otherwise benign site. The two basic types of XSS attacks are:
 - Reflected XSS
 - A reflected XSS attack exploits vulnerable Web applications that display input parameters back to the browser without checking for the presence of active content in them.

Security Threats: XSS

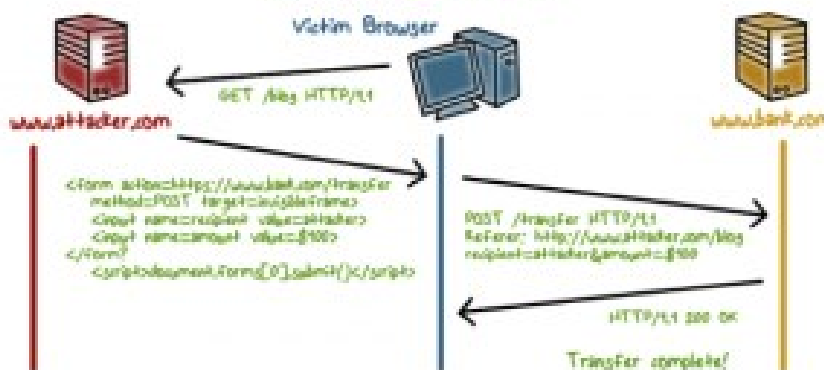
- Stored XSS
 - The stored XSS attack has become more important with the prevalence of Web 2.0. The key of Web 2.0 is sharing, interaction, and collaboration among people, so users have more chance of seeing other (potentially malicious) users' input through services such as social network services (SNS), wikis, or blogs.
- In either case, input value validation and sanitization are the key to preventing XSS attacks.
- Usually Web servers remove scripts from user input, but often attackers exploit vulnerabilities to bypass these filters, resulting in major attacks

Security Threats: Cross-site reference forgery

- Cross-site request forgery, also known as a one-click attack or session riding and abbreviated as CSRF ("sea-surf") or XSRF, is a type of malicious exploit of a website whereby unauthorized commands are transmitted from a user that the website trusts.
- Unlike cross-site scripting (XSS), which exploits the trust a user has for a particular site, CSRF exploits the trust that a site has in a user's browser.

Security Threats: Cross-site reference forgery

XSRF: Example



Security Threats: cookies

- A cookie is a small piece of text stored on a user's computer by a web browser.
- It is sent as an HTTP header by a web server to a web browser and then sent back unchanged by the browser each time it accesses that server.
- Being simple pieces of text, cookies are not executable. They are neither spyware or viruses, although cookies from certain sites are detected by many anti-spyware products because they can allow users to be tracked when they visit various sites

Security Threats: cookies

- Tracking cookies may be used to track internet users' web browsing habits. This can also be done in part by using the IP address of the computer requesting the page or the referer field of the HTTP header, but cookies allow for a greater precision.

Security Threats: some attacks

- Cookie Replay Attack
- Exploiting *Browser Cookies* to Bypass HTTPS connections
- Phishing with autofill
- ...and many others

Security and Privacy

- When seeking which browser to use, one of your primary concerns should be its security and how the developer is taking action against threats.
- Traditionally, Internet Explorer probably has the worst track record when it came to data breaches, with security holes exposed and taken advantage of regularly.
- But with the introduction of Microsoft newest browser, Edge, it promises to have overcome some of its problems from the past and introduced tools like Microsoft SmartScreen to prevent phishing attempts

Security and Privacy

- Firefox hasn't launched any fancy new tech to deal with security threats, it just has a tasty bounty programme that relies on a huge community to uncover potential security holes.
- Mozilla, the company behind Firefox, stated that "individuals security and privacy on the internet are fundamental and must not be treated as optional".
- Chrome is probably the most secure browser out there, but its privacy record is pretty questionable and that's why it's not the browser market leader.

Security and Privacy

- At Pwn2Own 2017, an event where hackers exploit devices to win prizes and show tech companies what patches they need to make:
 - Edge came off as the least secure browser, with hackers successfully exploiting it five times.
 - Safari was compromised 3.5 times, with judges awarding half a point for a vulnerability fixed in a beta version of the browser.
 - Firefox was beaten once.
 - Chrome was the most secure as it wasn't hacked at all.

Graphics

- MotionMark is a graphics benchmark that measures a browser's ability to animate complex scenes at a target frame rate. For this test, the bigger the score, the better the browser has performed.
- On the Acer laptop
 - Edge came first with a score of 151.23
 - Chrome at 147.55.
 - Firefox lagged behind since it received a score of 94.26.
- On the Macbook the scores were noticeably higher
 - Safari was ahead with a score of 455.20.
 - Chrome at 324.86
 - Firefox at 107.76.

Responsiveness of web applications

- Finally, we carried out the Speedometer benchmark that measures the responsiveness of web applications. It does its best to replay a typical workload and simulates adding, completing and removing to-do items.
- On the Acer laptop
 - Chrome scores 81.4 runs per minute
 - Firefox scores 34.6 runs per minute
 - Microsoft Edge 32.33.
- On the MacBook Pro,
 - Safari scores of 130 runs a minute
 - Chrome 116
 - Firefox 47.4

Mobile Compatibility

- To sync your Chrome accounts up, you just need to make sure you're signed into your Gmail account on both your desktop and your smartphone and everything will transfer - bookmarks, preferences and any other info you may have set up.
- It's a similar story with Firefox, although you'll need to set up an account if you want to use the same settings across devices. You'll just need to head to the settings menu on either desktop or mobile, then sync and enter your details if you have an account or create one if you don't.
- Microsoft Edge was the last of the three browsers to move to mobile, but that doesn't mean it's lagging behind. Of course, Edge isn't available on Macs, so it's likely only Windows 10 users will want to install the app on their iPhone or Android phone, but it still provides a solid experience, allowing you to carry on browsing the pages you were using on desktop, right from your mobile.

Java Script

- We used the Ares 6 test to measure the execution of JavaScript's newest features
- On the Acer laptop
 - Chrome stormed ahead with a time of 114.67ms.
 - Firefox had a time of 182.58ms.
 - Edge lagged way behind at 232.13ms
- On the MacBook
 - Safari finishing incredibly fast in 31.89ms
 - Chrome at 50.47ms
 - Firefox had a time of 121.34ms.

Java Script

- Jetstream, which measures a variety of JavaScript benchmarks. These focus on a variety of advanced workloads and programming techniques, and reports a single score. Bigger scores are better here.
- On Windows
 - Edge had a strong score:138.79ms
 - Firefox 98.844ms
 - Chrome 98.424ms
- On the MacBook
 - Safari scoring 181.09ms
 - Chrome 164.43ms
 - Firefox 142.52ms.

Java Script

- The last JavaScript test was the Octane benchmark, which measures a JavaScript engine's performance by running a range of tests which are representative of certain use cases in JavaScript applications.
- On Windows
 - Edge scored 20047
 - Chrome 18999
 - FireFox 17691
- On the MacBook
 - Chrome 29582
 - Safari 29253
 - Firefox 25204